

Multi-Agent System for Dynamic Retail Pricing with Edge-IoT Integration

RL Pricing Agent: Design Specification

Field	Detail
Author (this report)	Parth Gala (parth.gala@sjsu.edu)
Project Advisor	Prof. Vidyacharan Bhaskar, SJSU
Course	CMPE 295A, Master's Project, San Jose State University
Project Team	Parth Gala, Aruna Samhitha Mutyala, Het Bhalala, Rodrigo Chen
Document Version	0.2
Status	Work in Progress
Current Sprint	Sprint 3 (Mar 29 to Apr 11, 2026)

Note: I wrote this document to detail my individual work on the RL Pricing Agent. My teammates each have their own submissions covering the SHAP transparency layer (Aruna), ESP32 firmware (Het), supplier ingestion service (Rodrigo), and manager dashboard.

1. Version History

Version	Changes
0.1	Initial draft covering environment design, reward function, and DQN training setup
0.2	Added Sprint 2 DQN results, parameter sweep findings, and DDPG status update

2. Introduction

This document details my work on the RL Pricing Agent, which is my individual contribution to the project. The agent is part of Layer 4 (the AI Backend) of our 5-layer system and is responsible for computing price adjustments using two reinforcement learning algorithms: Deep Q-Network (DQN) and Deep Deterministic Policy Gradient (DDPG). At each step, it takes in a 4-dimensional observation of the current retail state and outputs a pricing decision, which then gets published to the MQTT broker and delivered to the ESP32 edge displays.

My work specifically covers building the RetailPricingEnv gym environment, setting up the DQN and DDPG training pipelines, designing and validating the reward function, and writing the MQTT publisher. This work sits alongside but does not overlap with what my teammates are building.

3. References

1. workbook_assignment_1.md, Ch. 8 (Implementation Plan) and Ch. 9 (Schedule), CMPE 295A GitHub, SJSU, 2026
 2. Multi-Objective Hierarchical RL with Online Meta-Learning for Dynamic Pricing. ResearchGate, 2026
 3. Multi-objective Linear RL with Lexicographic Rewards. ICML 2025
 4. Stable-Baselines3 Docs: <https://stable-baselines3.readthedocs.io>
 5. Farama Gymnasium Docs: <https://gymnasium.farama.org>
-

4. Requirements

ID	Requirement	Status
R1	State Space: 4D observation vector consisting of inventory_level in [0,1], competitor_price in [0,1], day_of_week in {0 to 6}, and time_of_day_bucket in {0 to 3}	[Done] Complete
R2	Action Space (DQN): 5 discrete price adjustment steps: -\$0.50, -\$0.25, \$0.00, +\$0.25, +\$0.50	[Done] Complete
R3	Action Space (DDPG): Continuous price delta ranging from -\$1.00 to +\$1.00, replacing the discrete action space used in DQN	[In Progress] In Progress
R4	Reward Function: Profit minus a volatility penalty (weighted by lambda=0.3) and a churn risk penalty (weighted by gamma=0.5). Penalties are turned off for the first 50 training episodes to allow the agent to stabilize.	[Done] Complete
R5	Latency: The full pipeline from agent decision to ESP32 display update must complete within 5 seconds (NFR-01)	[Pending] Pending, end-to-end MQTT test planned for Sprint 4
R6	MQTT Output: Price decisions published to retail/price/{product_id} at QoS 1, with a JSON payload containing product_id, price, delta, and timestamp	[In Progress] In Progress, local test passed but live integration still pending
R7	Training Infrastructure: SB3 DQN and DDPG training on the SJSU HPC GPU cluster, with model checkpoints saved every 50 episodes	[Pending] Pending, HPC workflow still being stabilized

5. Functional Overview

The core of my work is `RetailPricingEnv`, a custom environment that implements the Gymnasium interface. I built it to simulate the retail store so the RL agent can train by interacting with it. It has three main methods: `reset()` which sets up a fresh episode by sampling a starting state from the synthetic dataset, `step(action)` which applies the chosen price change, simulates the resulting demand, and calculates the reward, and `render()` which prints a readable summary of the current state for debugging.

For the reward function, I wanted the agent to do more than just maximize profit. A naive profit-only reward tends to produce erratic pricing that annoys customers. So the reward penalizes two things in addition to profit: how much the price moved compared to the last step (volatility), and how likely the current pricing is to cause a customer to stop buying (churn risk, predicted by Aruna's SHAP ensemble model). The penalty weights, $\lambda=0.3$ for volatility and $\gamma=0.5$ for churn, were validated empirically using a 9-point parameter sweep in Sprint 2.

To prevent the agent from getting stuck early in training, I added a curriculum warm-up phase using a custom `WarmUpCallback`. For the first 50 episodes, the penalty terms are turned off and the agent just optimizes for profit. This helps it learn a baseline pricing policy before the harder constraints kick in. Prof. Bhaskar reviewed this at the March 20 advisor meeting and confirmed it's a good approach worth highlighting in the paper.

The DQN agent uses the SB3 implementation with an epsilon-greedy exploration schedule that decays from 1.0 down to 0.05 over 500 episodes, a replay buffer of 50,000 transitions, and a target network that updates every 100 steps. After training for 1,000 episodes on the SJSU HPC cluster, it converged around episode 700 and achieved +22% average profit compared to a fixed-markup baseline, while also reducing how often it changed prices by 38%.

The DDPG agent extends this to a continuous action space. Instead of picking from 5 preset price steps, it outputs a continuous price delta anywhere between -\$1.00 and +\$1.00. I'm currently running the 1,000-episode training job on HPC and it's about 60% done as of Apr 8.

For the MQTT publisher, I used `paho-mqtt` to send a price decision message to the broker after each agent step. I've tested it locally with a Mosquitto broker and confirmed that messages arrive correctly. Connecting it to the live ESP32 subscriber is the remaining piece, planned for Sprint 4.

6. Configuration / External Interfaces

Software used: Python 3.11, PyTorch 2.x, Stable-Baselines3, Gymnasium, `paho-mqtt`, NumPy, pandas, TensorBoard.

Infrastructure: SJSU HPC GPU Cluster (SLURM scheduler, 12-hour job wall time) and a local Mosquitto MQTT broker for testing.

DQN hyperparameters finalized after Sprint 2:

`learning_rate` = $1e-3$
`epsilon schedule` = 1.0 down to 0.05 over 500 episodes
`replay_buffer_size` = 50,000

batch_size = 64
target_update_freq = every 100 steps
checkpoint_interval = every 50 episodes

Reward function parameters, selected via 9-point grid sweep:

lambda = 0.3 (volatility penalty weight)
gamma = 0.5 (churn risk penalty weight)
warm_up_episodes = 50

7. Debug

7.1 Logging

I set up three logging outputs to make training easy to monitor:

- TensorBoard: SB3 logs episode reward, loss, and Q-value estimates to ./logs/tb/ during training. Run tensorboard --logdir ./logs/tb/ to view the charts.
- CSV export: A custom BaseCallback writes one row per episode to results/metrics_log.csv with the columns episode, total_reward, avg_profit, price_change_count, epsilon, and q_value_mean.
- MQTT shadow topic: Each pricing decision is also published to retail/debug/pricing at QoS 0, with a verbose payload that includes the full state vector and the individual reward components (profit, volatility penalty, churn penalty). This makes it easy to inspect what the agent is doing without digging into the SB3 internals.

7.2 Counters

The following metrics are tracked per training run: episodes_trained, avg_episode_reward (rolling 100-episode window), price_change_frequency, q_value_mean, and convergence_episode.

8. Implementation

The work is split across five sprints. I've described each below with what was completed and any blockers I ran into.

8.1 Sprint 0 (pre-Mar 1) [Done]

- Set up the full development environment: Python 3.11, PyTorch, SB3, Gymnasium, and paho-mqtt in a virtual environment.
- Reviewed the relevant literature to decide on initial reward function weights; proposed lambda=0.3 and gamma=0.5 as starting values based on the MOHRL paper.
- Drafted and finalized Workbook Ch. 8 and Ch. 9, which I was responsible for, and coordinated the final Canvas submission on Mar 1.

- Submitted application for SJSU HPC GPU cluster access. This ended up taking 18 days to get approved, which became a bottleneck in Sprint 1.

8.2 Sprint 1 (Mar 1 to 14) [Done]

- Built the full `RetailPricingEnv` class: observation and action spaces, `reset()`, `step()`, and `render()`.
- Implemented and validated the `WarmUpCallback` for the curriculum learning phase.
- Ran the environment through the Gymnasium API checker to catch any interface issues before training.
- Since HPC access had not come through yet, I ran 200-episode smoke tests on local CPU just to confirm the environment logic was correct. Training speed was not a concern at this stage.

8.3 Sprint 2 (Mar 15 to 28) [Done]

- HPC access was finally granted on Mar 18. Restarted DQN training on GPU, which was significantly faster.
- Ran the full 1,000-episode DQN training job. The agent converged around episode 700.
- Implemented a simple rule-based baseline (fixed 30% markup) to compare against. Its average daily profit on the synthetic dataset was \$142.30.
- DQN results: +22% average profit and 38% fewer price changes compared to the baseline.
- Ran a 9-point parameter sweep (λ in {0.1, 0.3, 0.5} crossed with γ in {0.3, 0.5, 0.7}, 500 episodes each) to validate the reward weights. $\lambda=0.3$ and $\gamma=0.5$ came out on top, confirming the initial choice.
- Blocker: the HPC job got preempted at around episode 500 on Mar 23 due to a node failure. I recovered from the last checkpoint but this set me back about a day. To avoid losing progress again, I reduced the checkpoint interval from 200 down to 50 episodes.

8.4 Sprint 3 (Mar 29 to Apr 11) [In Progress] In Progress

- Refactored the action space in `RetailPricingEnv` from `Discrete(5)` to a continuous `Box` ranging from -1.0 to +1.0, which is what DDPG needs. The reward function did not change.
- Set up the SB3 DDPG actor-critic with Ornstein-Uhlenbeck noise and submitted the 1,000-episode training job to HPC. About 600 of 1,000 episodes are done as of Apr 8. Had another preemption on Apr 5 but recovered fine from the checkpoint.
- Added the `paho-mqtt` publisher to the agent's step loop and ran a local test with Mosquitto. Five messages were sent and received correctly with the expected JSON format.
- Still to finish: waiting for DDPG to converge, then writing up the DQN vs. DDPG comparison, and doing the first live test with Het's ESP32 subscriber.

8.5 Sprint 4 (Apr 12 to 25) [Pending] Upcoming

- Wire up the full end-to-end pipeline: agent decision to MQTT broker to ESP32 display update.
- Measure the round-trip latency and check it against the 5-second requirement.
- Write up the DQN vs. DDPG comparison in terms of final reward, how fast each converged, and the shape of their action distributions.

8.6 Sprint 5 (Apr 26 to May 9) [Pending] Upcoming

- Final system-level testing across all integrated components.
 - Write the RL agent section of the IEEE ICCE 2026 paper, including the reward function design rationale and the experimental results.
-

9. Testing

9.1 General Approach

I test at three levels. Unit tests cover RetailPricingEnv in isolation using pytest and make sure the core logic is correct before any training happens. Integration tests check that the agent and the MQTT broker talk to each other correctly. System tests will cover the full pipeline latency once Sprint 4 integration is done.

For the baseline comparison, both the DQN and DDPG agents will be evaluated on the same 30-day synthetic dataset with 10 SKUs and compared against the fixed-markup rule-based policy.

9.2 Unit Tests

ID	Test	Status
UT-01	reset() returns a state vector with all values within their declared ranges	[Done] Pass
UT-02	When warm-up is active (episode below 50), reward equals profit with no penalties applied	[Done] Pass
UT-03	When warm-up is inactive, reward equals profit minus 0.3 times volatility minus 0.5 times churn for a known input	[Done] Pass
UT-04	render() returns a non-empty string without throwing an exception	[Done] Pass
UT-05	After the DDPG refactor, the action space is a Box with shape (1,) ranging from -1.0 to +1.0	[Done] Pass
UT-06	MQTT publish produces valid JSON with the keys product_id, price, delta, and timestamp	[In Progress] In Progress

UT-07	A checkpoint file is written after 55 training episodes when the interval is set to 50	[Done] Pass
-------	--	-------------

10. Appendix

State and action space details:

Index	Feature	Range
0	inventory_level	0.0 to 1.0, normalized
1	competitor_price	0.0 to 1.0, normalized
2	day_of_week	0 (Monday) to 6 (Sunday)
3	time_of_day_bucket	0=morning, 1=afternoon, 2=evening, 3=night

DQN action mapping: 0 is -\$0.50, 1 is -\$0.25, 2 is \$0.00, 3 is +\$0.25, 4 is +\$0.50. DDPG action: continuous price delta from -\$1.00 to +\$1.00.

Reward function formula:

Reward = (price - cost) * units_sold
- 0.3 * abs(price_t - price_t-1)
- 0.5 * churn_model.predict_proba([state])[1]

Lambda and gamma sweep results, showing percent profit improvement over the rule-based baseline at episode 500:

lambda	gamma	0.3	0.5	0.7
0.1		+17%	+18%	+16%
0.3		+21%	+22%	+20%
0.5		+19%	+19%	+17%

Lambda=0.3 and gamma=0.5 (shown in the middle of the table) gave the best result at +22%, which matched my initial design choice from the literature review.